# COMMUNICATION WITH AI

## GRADE 8 • MONTH 2

## Chapter 2: Python Background

Coding a Solution

**Python + AI-themed learning**

```
name = input("Enter your name: ")\nif name:\n    print("Hello,", name)\nelse:\n    print("Please en
```

**What you will learn**

- What Python is and how programs run.
- How people communicate with AI using prompts and inputs.
- Python variables, data types, input, operators and conditions.
- How to write algorithms, flowcharts and simple Python programs.
- How to test, debug and improve a rule-based decision system.

**Student details**

Name: _____ Class: _____ Roll No.: _____

School: _____ Teacher: _____

## Contents

Your learning journey for Month 2


| Section | What it contains |
|---|---|
| Unit A: Introduction to Python | Python, programs, source code, interpreter, syntax and debugging |
| Unit B: Communicating with AI | Prompts, inputs, outputs, responsible AI communication |
| Unit C: Python Fundamentals | Variables, data types, input, operators, conditions, loops and lists |
| Unit D: Coding a Solution | Algorithms, flowcharts, five worked programs and testing |
| Classroom Activities | Five hands-on activities, including unplugged tasks |
| Worksheets | Three practice worksheets with writing space |
| MCQs, Quiz and Assessment | 25 MCQs, a 10-question quiz and a 30-mark assessment |
| Puzzles and Brain Games | Word search, matching, output prediction and debugging detective |

| Section | What it contains |
|---|---|
| Projects and Reflection | Case studies, mini-project, HOT questions, revision and glossary |
| Teacher Answer Key | Answers, explanations, code solutions and marking scheme |

# Unit A: Introduction to Python

Coding a Solution

## A1. What is Python?

**Python** is a popular high-level programming language. It is used to give computers clear instructions. Python is known for its simple, readable style, which makes it a good first language for students.

Python was created by **Guido van Rossum** and first released in **1991**. Its name was inspired by the comedy group **Monty Python**, not by the snake.

## A2. Why is Python popular?

- Its commands look similar to simple English.
- It can be used for websites, games, data analysis, automation and AI-related work.
- It has many free libraries, which are collections of ready-made code.
- It works on Windows, macOS and Linux.
- It is widely used in schools, universities and technology companies.

Python is often used with data and AI because it can process information, organise datasets, perform calculations and connect to AI tools. However, an ordinary Python program is not automatically an AI system. AI usually needs a trained model or an external AI service.

## A3. Where is Python used?

| Area | Example use of Python |
|---|---|
| Education | Checking quiz answers or creating a study planner |
| Healthcare | Organising appointment records or analysing health data with expert supervision |
| Agriculture | Recording rainfall and suggesting irrigation rules |
| Transportation | Calculating travel time or managing simple route information |
| Everyday life | Automating repeated tasks, such as making a shopping list |
| AI and data | Preparing data, performing calculations and using AI libraries or services |

## A4. Programs, languages and source code

A **program** is a set of instructions that tells a computer what to do. A **programming language**, such as Python, is the language we use to write those instructions. The written instructions are called **source code**.

Computers follow instructions exactly. They do not guess meanings. If an instruction is unclear or written incorrectly, the computer may show an error or produce an unexpected result.

| Human language | Programming language |
|---|---|
| “Please greet Riya politely.” | print("Hello, Riya!") |
| Flexible and context-based | Exact and rule-based |

| Human language | Programming language |
|---|---|
| Humans can infer meaning | Computers follow precise syntax |
| Small mistakes may still be understood | Small mistakes can cause errors |

## A5. How Python code runs

Python uses an **interpreter**. The interpreter reads Python source code line by line, translates it into instructions the computer understands, and runs it. If it finds a problem, it usually stops and displays an error message.

| Write Python code | Interpreter reads it | Computer follows instructions | Output appears |
|---|---|---|---|

## A6. Syntax, errors, debugging and output

**Syntax** means the correct grammar and structure of a programming language. **Output** is what the program shows after running. **Debugging** means finding and fixing mistakes in a program.

| Error type | Meaning | Example |
|---|---|---|
| Syntax error | The code breaks Python grammar rules | Missing bracket or colon |
| Runtime error | The code starts but fails while running | Dividing by zero |
| Logic error | The code runs but gives a wrong result | Using + instead of - |

**Debugging tip**
Read the error message carefully. It often tells you the line number and the type of problem. Check spelling, brackets, quotation marks, colons and indentation.

# Unit B: Communicating with AI

Instructions, prompts and responsibility

## B1. How people interact with AI

**Artificial Intelligence (AI)** refers to computer systems that can perform tasks often associated with human intelligence, such as recognising speech, answering questions or recommending videos. People communicate with AI by giving it information or instructions.

For example, a student may ask a voice assistant, “What is the capital of Japan?” The AI processes the spoken question and responds with an answer.

## B2. Prompts, inputs, outputs and responses

| Term | Meaning | Example |
|---|---|---|
| Prompt | An instruction or question given to an AI system | “Explain photosynthesis in simple words.” |
| Input | Data or information given to a system | Typed question, voice command, image or number |
| Processing | The system works on the input using rules, patterns or a trained model | The AI interprets the question |
| Output / response | The result produced by the system | A written explanation or spoken answer |

## B3. Everyday language vs Python instructions

When we communicate with an AI chatbot, we can use normal language. When we write a Python program, we must use exact Python syntax. Both need clear instructions, but Python is much stricter.

| AI chatbot prompt | Python instruction |
|---|---|
| “Say hello to Aman.” | print("Hello, Aman!") |
| “Add 8 and 5.” | print(8 + 5) |
| “Ask for my age.” | age = input("Enter your age: ") |
| “If the mark is 40 or more, say Pass.” | if mark >= 40: print("Pass") |

## B4. How a simple system processes information

| User gives input | System checks rules or patterns | System produces output | User checks the result |
|---|---|---|---|

In a simple Python program, the rules are written by a programmer. In a machine-learning AI system, patterns may be learned from data. A rule-based Python program is **not** machine learning unless it uses a trained model.

**Important distinction**

A rule-based program follows instructions written by a programmer.

A machine-learning system learns patterns from data.

Python can be used for both, but writing an if-else rule does not create AI by itself.

## B5. Clear, safe and responsible instructions

- Be specific: say exactly what you want.
- Give necessary context: mention your class, topic or format.
- Do not share private information, such as your full address, passwords, phone number or health records.
- Check AI answers with textbooks, teachers or trusted sources.
- Use AI to learn and practise—not to copy work without understanding.

## B6. AI communication in real life

| Field | AI communication example | Why checking matters |
|---|---|---|
| Education | A student asks AI to explain a maths step | The explanation may contain a mistake |
| Healthcare | A person asks about symptoms | Only a qualified health professional should diagnose |
| Agriculture | A farmer asks about crop disease | Local conditions and expert advice matter |
| Transportation | A navigation app suggests a route | Road closures or safety conditions may change |

# Unit C: Python Fundamentals

Variables, input, decisions and loops

## C1. print() function

The **print()** function displays information on the screen.

**Example 1: Using print()**

```
print("Hello, Grade 8!")
print("Python is fun.")
```

**Expected output:**

```
Hello, Grade 8!
Python is fun.
```

**Explanation:** The text inside quotation marks is a **string**. Python prints exactly what is inside the quotation marks.

**Common mistake:** Forgetting quotation marks: print(Hello) causes a NameError unless Hello is a variable.

**Practice:** Write a print() statement to display your favourite subject.

## C2. Comments

A **comment** is a note for humans. Python ignores comments when running the program. Comments begin with the # symbol.

**Example 2: Comments**

```
# This program greets the user
print("Hello!")  # This prints a greeting
```

**Expected output:** Hello!

**Common mistake:** Using comments to explain wrong code does not fix the code. Comments help people understand the program.

**Practice:** Add a comment above a print() statement explaining what it does.

## C3. Variables and assignment

A **variable** is a named container that stores a value. The **=** symbol assigns a value to a variable.

**Example 3: Variables**

```
name = "Asha"
age = 13
print(name)
print(age)
```

**Expected output:**

```
Asha
13
```

**Explanation: name** stores the text “Asha”. **age** stores the number 13. Variable names should be meaningful and cannot begin with a number.

**Common mistake:** Writing Print instead of print. Python is case-sensitive.

**Practice:** Create variables for your city and favourite game, then print them.

## C4. Data types

| Data type | Meaning | Example |
|---|---|---|
| Integer (int) | Whole numbers | age = 13 |
| Floating-point (float) | Decimal numbers | price = 25.5 |
| String (str) | Text in quotation marks | city = "Delhi" |
| Boolean (bool) | True or False | is_present = True |

**Example 4: Data types**

```
age = 13
height = 1.52
name = "Ravi"
is_present = True
print(type(age), type(height), type(name), type(is_present))
```

**Expected output:**

```
<class 'int'> <class 'float'> <class 'str'> <class 'bool'>
```

**Common mistake:** Writing true instead of True. Boolean values must begin with a capital letter.

**Practice:** Create one variable of each data type and print it.

## C5. Taking input with input()

The **input()** function asks the user to type something. It always returns a **string**.

**Example 5: User input**

```
name = input("Enter your name: ")
print("Welcome,", name)
```

**Sample run:**

```
Enter your name: Meera
Welcome, Meera
```

**Common mistake:** Trying to do maths directly with input. Convert it first using int() or float().

**Practice:** Ask for a user’s favourite colour and print a sentence using it.

## C6. Converting input

**Example 6: Conversion**

```
age = int(input("Enter your age: "))
height = float(input("Enter your height in metres: "))
print("Next year you will be", age + 1)
```

**Sample run:**

```
Enter your age: 13
Enter your height in metres: 1.5
Next year you will be 14
```

**Explanation:** int() converts text to a whole number. float() converts text to a decimal number.

**Common mistake:** Entering letters when the program expects a number. This causes a ValueError.

**Practice:** Ask for two whole numbers and print their sum.

## C7. Arithmetic operators

| Operator | Meaning | Example | Result |
|---|---|---|---|
| + | Addition | 7 + 2 | 9 |
| - | Subtraction | 7 - 2 | 5 |
| * | Multiplication | 7 * 2 | 14 |
| / | Division | 7 / 2 | 3.5 |
| // | Floor division | 7 // 2 | 3 |
| % | Remainder | 7 % 2 | 1 |
| ** | Power | 7 ** 2 | 49 |

### Example 7: Arithmetic operators

```
a = 10
b = 3
print(a + b, a - b, a * b, a / b, a // b, a % b, a ** b)
```

**Expected output:** 13 7 30 3.3333333333333335 3 1 1000

**Common mistake:** Using = instead of == when comparing values.

**Practice:** Calculate the area of a rectangle with length 8 and width 5.

## C8. Comparison and logical operators

| Operator | Meaning | Example | Result |
|---|---|---|---|
| == | Equal to | 5 == 5 | True |
| != | Not equal to | 5 != 3 | True |
| > | Greater than | 5 > 8 | False |
| < | Less than | 5 < 8 | True |
| >= | Greater than or equal to | 5 >= 5 | True |
| <= | Less than or equal to | 5 <= 4 | False |
| and | Both conditions are True | 5 > 3 and 2 > 1 | True |
| or | At least one condition is True | 5 < 3 or 2 > 1 | True |
| not | Reverses the result | not(5 > 3) | False |

**Practice:** Predict the result of 10 >= 10 and 4 != 4.

## C9. Conditional statements: if, elif, else

Conditional statements let a program make decisions.

**Example 8: if-else**

```
marks = int(input("Enter your marks: "))

if marks >= 40:
    print("Pass")
else:
    print("Try again")
```

**Sample run:**

```
Enter your marks: 65
Pass
```

**Explanation:** Python checks whether marks is 40 or more. If yes, it prints Pass. Otherwise, it prints Try again. The indented lines belong to the condition.

**Example 9: if-elif-else**

```
temperature = float(input("Enter temperature in °C: "))

if temperature < 20:
    print("Cool")
elif temperature <= 30:
    print("Warm")
else:
    print("Hot")
```

**Sample run:**

```
Enter temperature in °C: 27
Warm
```

**Common mistake:** Forgetting the colon (:) or incorrect indentation. Both cause syntax errors.

**Practice:** Write a program that prints “Even” if a number is divisible by 2, otherwise “Odd”.

## C10. Loops

A loop repeats instructions. A **for** loop repeats a known number of times. A **while** loop repeats while a condition remains True.

**Example 10: for loop**

```
for i in range(1, 6):
    print("Revision day", i)
```

**Expected output:**

```
Revision day 1
Revision day 2
Revision day 3
Revision day 4
Revision day 5
```

**Example 11: while loop**

```
count = 1
while count <= 3:
    print("Count is", count)
    count = count + 1
```

**Expected output:**

```
Count is 1
Count is 2
Count is 3
```

**Common mistake:** Forgetting to increase count in a while loop. This creates an infinite loop.

**Practice:** Use a for loop to print the 2 times table from 2 to 20.

## C11. Lists and simple datasets

A **list** stores several values in one variable.

**Example 12: Lists**

```
fruits = ["apple", "banana", "lime"]
print(fruits[0])
print(fruits[2])
fruits.append("mango")
print(fruits)
```

**Expected output:**

```
apple
lime
['apple', 'banana', 'lime', 'mango']
```

**Explanation:** List positions start from 0. fruits[0] is the first item. append() adds an item at the end.

**Common mistake:** Asking for fruits[3] when the list has only three items. This causes an IndexError.

**Practice:** Create a list of four school subjects and print the first and last items.

## C12. String handling

**Example 13: Strings**

```
name = "grade8"
print(name.upper())
print(len(name))
print(name[0])
```

**Expected output:**

```
GRADE8
6
g
```

**Practice:** Ask for a user's name and print it in capital letters.

## C13. Writing readable code

- Use meaningful variable names, such as total_marks instead of x.
- Indent code consistently using four spaces inside if, for and while blocks.

- Add short comments to explain difficult steps.
- Test programs with normal, large, small and unusual inputs.
- Fix one error at a time and run the program again.

# Unit D: Coding a Solution

From problem to Python program

## D1. Problem-solving steps

| Identify problem | Understand input | Plan processing | Decide output | Write algorithm | Draw flowchart | Write code | Test and debug |
|---|---|---|---|---|---|---|---|

## D2. Algorithms and pseudocode

An **algorithm** is a step-by-step method to solve a problem. **Pseudocode** is a rough plan written in simple English-like steps; it is not exact Python code.

**Example 14: Pseudocode**

```
START
Ask for the student's marks
If marks are 40 or more
    Display "Pass"
Otherwise
    Display "Try again"
END
```

## D3. Input-Process-Output diagram

| Input | Process | Output |
|---|---|---|
| Marks | Compare marks with 40 | Pass or Try again |
| Two numbers and an operation | Perform calculation | Result |
| Temperature | Compare with temperature rules | Cool, Warm or Hot |

## D4. Worked Program 1: Greeting a user

**Problem:** Ask for a user's name and greet them.

**Algorithm:** 1. Ask for the name. 2. Store it in a variable. 3. Display a greeting.

**Program 1**

```
name = input("Enter your name: ")
print("Hello,", name + "!")
print("Welcome to Python.")
```

**Sample input/output:**

```
Enter your name: Aditi
Hello, Aditi!
Welcome to Python.
```

**Line-by-line explanation:** Line 1 asks for input and stores it in name. Line 2 prints a greeting using the stored name. Line 3 prints a welcome message.

**Test cases:** “Ravi” → Hello, Ravi!; “Sara” → Hello, Sara!.

**Improvement:** Remove extra spaces from the input using name.strip().

## D5. Worked Program 2: Simple calculator

**Problem:** Add, subtract, multiply or divide two numbers.

**Algorithm:** 1. Ask for two numbers. 2. Ask for the operation. 3. Perform the correct calculation. 4. Display the result.

**Program 2**

```
a = float(input("Enter first number: "))
b = float(input("Enter second number: "))
operation = input("Enter operation (+, -, *, /): ")

if operation == "+":
    print("Result:", a + b)
elif operation == "-":
    print("Result:", a - b)
elif operation == "*":
    print("Result:", a * b)
elif operation == "/":
    if b != 0:
        print("Result:", a / b)
    else:
        print("Cannot divide by zero")
else:
    print("Unknown operation")
```

**Sample input/output:**

```
Enter first number: 12
Enter second number: 4
Enter operation (+, -, *, /): /
Result: 3.0
```

**Test cases:** 8 and 2 with + → 10.0; 10 and 0 with / → Cannot divide by zero; 5 and 3 with % → Unknown operation.

**Improvement:** Repeat the calculator using a loop until the user chooses to stop.

## D6. Worked Program 3: Temperature classifier

**Problem:** Classify a temperature as Cool, Warm or Hot.

**Rules:** Below 20°C = Cool; 20°C to 30°C = Warm; above 30°C = Hot.

**Program 3**

```
temperature = float(input("Enter temperature in °C: "))

if temperature < 20:
    print("Cool")
elif temperature <= 30:
    print("Warm")
else:
    print("Hot")
```

**Sample input/output:**

```
Enter temperature in °C: 18
Cool
```

**Test cases:** 19 → Cool; 20 → Warm; 30 → Warm; 31 → Hot.

**Improvement:** Add a range check for impossible temperatures, such as below -90°C.

## D7. Worked Program 4: Student study helper

**Problem:** Recommend a revision activity based on study hours.

**Rules:** 3 or more hours → practise questions; 1 to less than 3 hours → revise notes; less than 1 hour → start a short session.

**Program 4**

```
hours = float(input("How many hours did you study today? "))

if hours >= 3:
    print("Great work! Try practice questions.")
elif hours >= 1:
    print("Revise today's notes and make a summary.")
else:
    print("Start with a 20-minute revision session.")
```

**Sample input/output:**

```
How many hours did you study today? 1.5
Revise today's notes and make a summary.
```

**Test cases:** 3 → practice questions; 2 → revise notes; 0.5 → short session.

**Improvement:** Add subject input and suggest a subject-specific activity.

## D8. Worked Program 5: Recycling classifier

**Problem:** Tell a user which bin to use for an item. This is a **rule-based program**, not machine learning.

**Rules:** Plastic bottle → recycling bin; paper → paper bin; banana peel → compost; other → ask an adult.

**Program 5**

```
item = input("Enter an item: ").lower()

if item == "plastic bottle":
    print("Put it in the recycling bin.")
elif item == "paper":
    print("Put it in the paper bin.")
elif item == "banana peel":
    print("Put it in the compost bin.")
else:
    print("Ask a teacher or adult how to dispose of it.")
```

**Sample input/output:**

```
Enter an item: PAPER
Put it in the paper bin.
```

**Test cases:** “plastic bottle” → recycling bin; “banana peel” → compost; “battery” → ask an adult.

**Improvement:** Add more items and use a list of acceptable recycling items.

### Rule-based vs machine learning

Rule-based program: A programmer writes every decision rule.

Machine-learning system: The system learns patterns from labelled data.

This chapter focuses on rule-based Python programs.

# Classroom Activities

Learn by doing

## Activity 1: Human Instructions vs Python Instructions

**Learning objectives**

Compare everyday instructions with exact Python instructions.

Understand why computers need precise syntax.

**Materials:** Paper, pencils, printed instruction cards.

**Preparation:** Write tasks such as “Make a paper aeroplane” and “Greet a friend.”

**Steps:**

- In pairs, write everyday instructions for making a paper aeroplane.
- Rewrite the same task as numbered, exact steps.
- Discuss what happens if a step is missing or unclear.
- Compare the instructions with this Python command: print("Hello!")
- Identify what a computer cannot guess.

**Student task:** Complete the table.

| Everyday instruction | Exact step-by-step instruction | What could go wrong? |
|---|---|---|
| | | |
| | | |
| | | |
| | | |

**Expected outcome:** Students explain that computers need complete, ordered and precise instructions.

**Discussion questions:** Why can a human understand an unclear instruction more easily than a computer? What details did you need to add?

**Extension:** Write instructions for drawing a square using only “move forward” and “turn right.”

**Teacher guidance:** Emphasise sequence, precision and testing. Accept reasonable written instructions.

**Assessment criteria:** Clear steps, correct order, identification of missing details.

## Activity 2: Build Your First Python Program

**Learning objectives**
Use print(), variables and input() to create a simple program.

**Materials:** Computer with Python 3, or paper for an unplugged version.

**Steps:**

- Type or write: name = input("Enter your name: ")
- Add: print("Hello,", name)
- Run the program and enter your name.
- Change the greeting message.
- Add a second question about your favourite subject.

**Activity code**

```
name = input("Enter your name: ")
subject = input("Enter your favourite subject: ")
print("Hello,", name)
print("You enjoy", subject)
```

**Expected outcome:** A program that asks for two inputs and prints a personalised message.

**Discussion questions:** What happens if you forget quotation marks? Why does input() always give text?

**Extension:** Ask for the user's age and print it after one year using int().

**Teacher guidance:** Check indentation is not needed here, but reinforce exact spelling of print and input.

**Assessment criteria:** Correct syntax, correct input/output, personalised message.

## Activity 3: Python Input-Process-Output Challenge

**Learning objectives**
Identify input, processing and output for simple problems.

**Materials:** Worksheet or notebooks.

**Steps:**

- Read each problem scenario.
- Identify the input data.
- Write the processing steps.
- State the expected output.
- Draw an arrow diagram: Input → Process → Output.

**Student task:** Complete the table.

| Problem | Input | Process | Output |
|---|---|---|---|
| Calculate the total of two test scores | | | |
| Decide whether a number is even or odd | | | |
| Recommend a book based on reading level | | | |
| Convert temperature to Cool/Warm/Hot | | | |

**Expected outcome:** Students can separate data, processing rules and results.

**Discussion questions:** Which problems need a decision? Which need a calculation?

**Extension:** Add one new input to each problem and explain how the process changes.

**Teacher guidance:** Accept clear, logical answers. Encourage precise rules.

**Assessment criteria:** Correct identification of input, process and output.

## Activity 4: Debug the Broken Code

**Learning objectives**
Find and correct syntax, indentation and logic errors.

**Materials:** Printed code cards or computers.

**Steps:**

- Read each program carefully.
- Identify the type of error.
- Correct the code.
- Predict the output after correction.
- Test with the given inputs.

**Buggy code 1**

```
# Debug this program
name = input("Enter your name: "
print("Hello, " name)

if 5 > 3
print("Five is greater")
```

**Student task:** Write the corrected program.

**Buggy code 2**

```
# Debug this program
marks = int(input("Enter marks: "))

if marks >= 40:
print("Pass")
else:
print("Try again")
```

**Student task:** Identify the error type and correct it.

**Expected outcome:** Students correct missing brackets, commas, colons and indentation.

**Discussion questions:** What is the difference between a syntax error and a logic error? Why does Python use indentation?

**Extension:** Create your own buggy program for a classmate to debug.

**Teacher guidance:** Ask students to explain the error before correcting it.

**Assessment criteria:** Correct identification, correction and explanation.

## Activity 5: Design and Code a Simple AI-Inspired Decision System

**Learning objectives**
Design a rule-based decision system, draw a flowchart and write Python code.

**Materials:** Paper, pencils, optional computer with Python 3.

**Scenario:** Design a system that recommends a school activity based on a student's interest.

**Steps:**

- Choose a problem, such as recommending a sport, book or revision activity.
- Identify the input, such as interest or time available.
- Write at least three decision rules.
- Draw a labelled flowchart.
- Write a Python program using if, elif and else.
- Test with at least three inputs.
- Discuss one limitation and one ethical concern.

**Sample solution**

```
interest = input("Enter your interest (science/sport/art): ").lower()

if interest == "science":
    print("Try a simple experiment or science quiz.")
elif interest == "sport":
    print("Join a football or athletics practice.")
elif interest == "art":
    print("Create a poster about an environmental issue.")
else:
    print("Choose an activity with your teacher.")
```

**Expected outcome:** A working rule-based program with a flowchart and test results.

**Discussion questions:** Is this real AI? Why or why not? What inputs could make the recommendation better?

**Extension:** Add a second input, such as available time.

**Teacher guidance:** Remind students this is rule-based, not machine learning.

**Assessment criteria:** Clear rules, correct flowchart, working code, testing and reflection.

# Worksheets

Practice and apply

## Worksheet 1: Python Background and Programming Basics

**A. Fill in the blanks.**

1. Python was created by _____.
2. A set of instructions for a computer is called a _____.
3. The _____ reads and runs Python code.
4. Notes for humans in Python begin with _____.
5. Finding and fixing errors is called _____.

**B. Match the following.**

| Column A | Column B |
|---|---|
| 1. Source code | a. Written Python instructions |
| 2. Interpreter | b. A mistake in a program |
| 3. Syntax | c. Runs Python code |
| 4. Bug | d. Grammar rules of a language |
| 5. Output | e. What a program displays |

**C. True or False.**

1. Python is a programming language. (_____)
2. Computers can guess unclear instructions. (_____)
3. Comments are ignored by Python. (_____)
4. Python is case-sensitive. (_____)
5. Every Python program is an AI system. (_____)

**D. Short answer.**

6. What is Python?

7. Give two reasons why Python is popular.

8. What is the difference between a program and source code?

9. What does an interpreter do?

10. Name three types of programming errors.

11. Why should AI outputs be checked?

12. What is a prompt?

13. Give one example of responsible AI use at school.

14. What personal information should you not share with an AI tool?

15. Is a rule-based Python program the same as machine learning? Explain.

# Worksheet 2: Python Syntax, Variables, Input and Output

**A. Predict the output.**

**Question 1**

```
print("AI Basics")
print("Month 2")
```

**Question 2**

```
x = 7
y = 3
print(x + y)
print(x * y)
```

**Question 3**

```
name = "Asha"
print("Hello,", name)
```

**Question 4**

```
print(10 > 4)
print(10 == 4)
```

**Question 5**

```
fruits = ["apple", "banana"]
print(fruits[1])
```

**B. Identify the data type.**

6. age = 13 _____

7. price = 19.5 _____

8. city = "Mumbai" _____

9. is_present = True _____

**C. Complete the code.**

10. Ask for the user's name and print a welcome message.

11. Ask for two whole numbers and print their sum.

12. Print “Pass” if marks are 40 or more, otherwise print “Try again”.

**D. Find and correct the error.**

**Question 13**

```
print("Hello World'
age = input("Enter age: ")
print(age + 1)
```

14. What error will occur if the user enters “ten” for age in the corrected program?

15. Rewrite the program so it works correctly for a whole-number age.

# Worksheet 3: Algorithms, Flowcharts and Coding a Solution

### A. Input-Process-Output.

| Problem | Input | Process | Output |
|---|---|---|---|
| Find the average of three marks | | | |
| Decide whether a student passes | | | |
| Classify recycling items | | | |

### B. Write an algorithm.

1. Write an algorithm to find the larger of two numbers.

2. Write an algorithm to decide whether a number is even or odd.

### C. Flowchart interpretation.

3. Draw a flowchart for: input a number; if it is positive, print “Positive”; otherwise print “Not positive”.

### D. Debugging.

**Question 4**

```
temperature = input("Enter temperature: ")

if temperature < 20:
print("Cool")
elif temperature <= 30:
print("Warm")
else
print("Hot")
```

4. Identify all errors and write the corrected program.

**E. Program design.**

5. A school wants a program to recommend a revision activity based on study hours. Write the rules, algorithm and Python code.

# MCQs, Quiz and Assessment

Check your understanding

## Multiple-Choice Questions (25 marks)

Choose the correct option. Each question carries 1 mark.

**1. Python was created by —**

   (a) Dennis Ritchie   (b) Guido van Rossum   (c) James Gosling   (d) Bill Gates

**2. Python was first released in —**

   (a) 1981   (b) 1991   (c) 2001   (d) 2011

**3. A set of instructions for a computer is called a —**

   (a) Dataset   (b) Program   (c) Flowchart   (d) Variable

**4. Python code that we write is called —**

   (a) Output   (b) Source code   (c) Hardware   (d) Machine code

**5. Which function displays text on the screen?**

   (a) input()   (b) print()   (c) if()   (d) for()

**6. Which symbol starts a comment in Python?**

   (a) //   (b) #   (c) <!--   (d) /*

**7. Which operator assigns a value?**

   (a) ==   (b) =   (c) !=   (d) >=

**8. Which data type stores 13?**

   (a) String   (b) Integer   (c) Boolean   (d) Float

**9. Which data type stores 2.5?**

   (a) Integer   (b) Boolean   (c) Float   (d) String

**10. Which data type stores True or False?**

   (a) Integer   (b) String   (c) Boolean   (d) Float

**11. What does input() return?**

   (a) Integer   (b) Float   (c) String   (d) Boolean

**12. Which function converts "13" to 13?**

   (a) str()   (b) int()   (c) float()   (d) print()

**13. What is 7 % 2?**

   (a) 3   (b) 1   (c) 3.5   (d) 14

**14. What is 2 ** 3?**

   (a) 6   (b) 8   (c) 9   (d) 5

**15. Which operator checks equality?**

   (a) =   (b) ==   (c) !=   (d) >=

**16. Which statement makes a decision?**

   (a) print()   (b) if   (c) input()   (d) for

**17. Which indentation is correct inside an if statement?**

   (a) Four spaces   (b) No spaces   (c) Random spaces   (d) Only tabs

**18. What does range(1, 4) produce?**

   (a) 1, 2, 3   (b) 1, 2, 3, 4   (c) 0, 1, 2, 3   (d) 1, 4

**19. Which loop repeats while a condition is True?**

   (a) for   (b) while   (c) if   (d) print

**20. Which is a valid list?**

   (a) (1, 2, 3)   (b) [1, 2, 3]   (c) {1, 2, 3}   (d) <1, 2, 3>

**21. What is fruits[0] for ["apple", "banana"]?**

   (a) banana   (b) apple   (c) error   (d) none

**22. What is a prompt?**

   (a) An AI output   (b) An instruction or question given to AI   (c) A Python error   (d) A variable name

**23. A rule-based Python program is —**

   (a) Always machine learning   (b) Not necessarily machine learning   (c) Always an AI model   (d) Never useful

**24. Which is a syntax error?**

   (a) Using a colon after if   (b) Forgetting a colon after if   (c) Printing the wrong value   (d) Using a variable

**25. Why should we test a program with several inputs?**

   (a) To make it slower   (b) To find errors and check rules   (c) To delete variables   (d) To avoid comments

## Quick Quiz: 10 Questions

**1. What does a program do?**

**2. What symbol begins a Python comment?**

**3. What does print() do?**

**4. What is a variable?**

**5. What data type is 3.14?**

**6. What does input() return?**

**7. Which keyword makes a decision?**

**8. What does a for loop do?**

**9. What is debugging?**

**10. Is a rule-based Python program always AI?**

## Chapter Assessment: 30 Marks

**Section A: Multiple Choice (1 mark each = 5 marks)**

1. Python is a: (a) programming language (b) database (c) robot (d) website

2. A comment begins with: (a) # (b) // (c) <!-- (d) **

3. input() returns a: (a) string (b) float only (c) Boolean (d) list

4. Which is a conditional statement? (a) for (b) if (c) print (d) input

5. A flowchart shows: (a) the steps of a solution (b) only output (c) only errors (d) hardware parts

**Section B: Short Answer (2 marks each = 10 marks)**

1. Define an algorithm.

2. What is pseudocode?

3. Differentiate syntax error and logic error.

4. What is a variable? Give one example.

5. Explain the difference between a rule-based program and machine learning.

**Section C: Application (3 marks each = 9 marks)**

1. Predict the output:

```
x = 6
y = 2
print(x * y)
print(x > y)
```

2. Correct the following code:

```
if 5 > 3
print("Yes")
```

3. Write a Python program to ask for a temperature and print Cool if it is below 20, otherwise print Warm or Hot.

**Section D: Higher-Order Thinking (6 marks)**

1. Design a rule-based program to recommend a recycling action for plastic bottles, paper and food waste. Write the rules and Python code. (4 marks)

2. A program has correct syntax but gives the wrong answer. What type of error is this? How can testing help? (2 marks)

**Total: 5 + 10 + 9 + 6 = 30 marks**

# Python Puzzles and Brain Games

Coding can be fun

## Puzzle 1: Python Vocabulary Word Search

Find: PYTHON, VARIABLE, INPUT, OUTPUT, LOOP, STRING, INTEGER, DEBUG, SYNTAX, FUNCTION.

| | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| P | Y | T | H | O | N | X | A | B | C |
| V | A | R | I | A | B | L | E | D | E |
| I | N | P | U | T | O | U | T | P | U |
| L | O | O | P | S | T | R | I | N | G |
| I | N | T | E | G | E | R | Y | X | A |
| D | E | B | U | G | F | U | N | C | T |
| S | Y | N | T | A | X | I | O | N | I |
| F | U | N | C | T | I | O | N | L | O |
| P | Y | T | H | O | N | X | A | B | C |
| V | A | R | I | A | B | L | E | D | E |

**Solution:** PYTHON (row 1), VARIABLE (row 2), INPUT (row 3), LOOP (row 4), STRING (row 4), INTEGER (row 5), DEBUG (row 6), SYNTAX (row 7), FUNCTION (row 8).

## Puzzle 2: Match Python Terms with Their Meanings

| Term | Meanings |
|---|---|
| 1. Variable | A. Repeats code |
| 2. String | B. Stores a value |
| 3. Loop | C. Text in quotation marks |
| 4. Boolean | D. Displays output |
| 5. print() | E. True or False |

**Solution:** 1-B, 2-C, 3-A, 4-E, 5-D.

## Puzzle 3: Predict the Output Challenge

**Predict the output**

```
a = 4
b = 3
print(a * b)
print(a > b)
print("Score:", a + b)
```

**Solution:** 12, True, Score: 7.

## Puzzle 4: Debugging Detective

**Find the errors**

```
age = input("Enter your age: ")
if age >= 13
print("Teenager")
else
print("Not teenager")
```

**Clues:** Check conversion, colon and indentation.

**Solution:** Convert input to int, add colons after if and else, and indent the print lines.

# Additional Learning Resources

Case studies, project and reflection

## Case Study 1: Student Study Planner

**Problem:** Students need help deciding how to revise after school.

**Input:** Study hours completed and upcoming test date.

**Processing rules:** If study hours are 3 or more, suggest practice questions. If 1 to less than 3, suggest note revision. If less than 1, suggest a short 20-minute session.

**Output:** A revision recommendation.

**Limitations:** The program does not know the student's subjects, strengths or exam difficulty.

**Questions:** What other input could improve the recommendation? Is this machine learning? Why or why not?

## Case Study 2: Recycling Awareness System

**Problem:** Students are unsure which bin to use for waste items.

**Input:** Name of the waste item.

**Processing rules:** Plastic bottle → recycling; paper → paper bin; banana peel → compost; unknown item → ask an adult.

**Output:** The correct bin or a safety message.

**Limitations:** The program only knows the items written into its rules.

**Questions:** Why is "ask an adult" a safe output for unknown items? How could the rules be improved responsibly?

## Mini-Project: Design and Code a Simple AI-Inspired Decision System

| Project step | Your response |
|---|---|
| 1. Real-life problem | |
| 2. Intended users | |
| 3. Input data | |
| 4. Decision rules | |
| 5. Algorithm in plain English | |
| 6. Labelled flowchart | |
| 7. Python program | |
| 8. Three test inputs and outputs | |
| 9. Limitation or ethical concern | |
| 10. Improvement | |

**Sample solution:** A program recommends a revision activity based on study hours. Input: hours studied. Rules: 3+ hours → practice questions; 1–3 hours → revise notes; below 1 hour → short session. Limitation: it does not know the subject or student's learning needs. Improvement: add subject and test-date inputs.

**Assessment rubric:** Problem clarity (2), suitable inputs/rules (3), algorithm and flowchart (3), correct Python code (4), testing (3), limitation and improvement (3). Total: 18 marks.

## Higher-Order Thinking Questions

1. Why must a computer program receive clear instructions?

2. Can a program produce a wrong result even if its syntax is correct? Explain.

3. How is a rule-based decision program different from a machine-learning system?

4. Why should a programmer test several inputs?

5. How could a decision program produce unfair results?

6. What personal information should a student avoid entering into an untrusted AI tool?

## Revision and Reflection

**Chapter summary:** Python is a programming language used to write instructions for computers. Programs take input, process it using rules, and produce output. Programmers use algorithms, flowcharts, variables, conditions, loops and testing to solve problems. A rule-based Python program is not automatically an AI system; AI may require trained models or external AI tools.

**Concept map:**

| Input | Algorithm / rules | Python code | Processing | Output | Testing and debugging |
|---|---|---|---|---|---|

**Revision exercises:**

- Write the difference between a syntax error and a logic error.

- Write a Python program to print your name five times.
- Draw an input-process-output diagram for a calculator.
- Explain why indentation matters in Python.
- Give two examples of responsible AI communication.

**Student reflection:**

1. Three new things I learned:

2. One question I still have:

3. One way I can use Python or AI responsibly:

## Self-Assessment Checklist

| I can… | Yes | Not yet |
|---|---|---|
| Explain what Python is | ■ | ■ |
| Use print() and comments | ■ | ■ |
| Create variables and identify data types | ■ | ■ |
| Use input(), int() and float() | ■ | ■ |
| Use if, elif and else | ■ | ■ |
| Use for and while loops | ■ | ■ |
| Create and use a list | ■ | ■ |
| Write an algorithm and flowchart | ■ | ■ |
| Debug simple Python programs | ■ | ■ |
| Explain responsible AI communication | ■ | ■ |

## Glossary

| Term | Meaning |
|---|---|
| Python | A high-level programming language |
| Program | A set of instructions for a computer |
| Source code | Written instructions in a programming language |
| Interpreter | A program that reads and runs Python code |

| Term | Meaning |
|---|---|
| Syntax | Grammar rules of a programming language |
| Error | A problem that prevents correct program behaviour |
| Debugging | Finding and fixing errors |
| Output | Result displayed by a program |
| Comment | A note for humans, ignored by Python |
| Variable | A named container that stores a value |
| Assignment | Storing a value in a variable using = |
| Integer | Whole-number data type |
| Float | Decimal-number data type |
| String | Text data type |
| Boolean | True or False data type |
| Input | Data entered into a program |
| input() | Python function that reads user input |
| int() | Converts text to an integer |
| float() | Converts text to a decimal number |
| Operator | A symbol that performs an action |
| Conditional statement | A statement that makes a decision |
| Loop | A structure that repeats code |
| List | An ordered collection of values |
| Algorithm | Step-by-step method to solve a problem |
| Pseudocode | Simple English-like plan for a program |
| Flowchart | A diagram showing the steps of a process |
| Prompt | Instruction or question given to an AI system |
| Machine learning | A method where AI learns patterns from data |
| Rule-based program | A program that follows rules written by a programmer |
| Responsible AI | Using AI safely, fairly and ethically |

# Teacher Answer Key

For teacher use only

## Worksheet 1: Python Background and Programming Basics

**A.** 1. Guido van Rossum 2. Program 3. Interpreter 4. # 5. Debugging.

**B.** 1-a, 2-c, 3-d, 4-b, 5-e.

**C.** 1. True 2. False 3. True 4. True 5. False.

**D.** 6. Python is a high-level programming language used to give computers instructions. 7. Simple syntax; useful for data, automation and AI-related work; many libraries; cross-platform. 8. A program is the set of instructions; source code is the written Python instructions. 9. It reads Python code and runs it. 10. Syntax, runtime and logic errors. 11. AI can make mistakes or give incomplete information. 12. An instruction or question given to an AI system. 13. Use AI to explain a concept, then check with a teacher or textbook. 14. Passwords, address, phone number, health records, private photos. 15. No. A rule-based program follows programmer-written rules; machine learning learns patterns from data.

## Worksheet 2: Python Syntax, Variables, Input and Output

**A.** 1. AI Basics / Month 2. 2. 10 / 21. 3. Hello, Asha. 4. True / False. 5. banana.

**B.** 6. Integer 7. Float 8. String 9. Boolean.

**C. 10.**

```
name = input("Enter your name: ")
print("Welcome,", name)
```

**11.**

```
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))
print("Sum:", a + b)
```

**12.**

```
marks = int(input("Enter marks: "))
if marks >= 40:
    print("Pass")
else:
    print("Try again")
```

**D. 13.** Missing closing quotation mark and bracket; missing comma in print; input returns a string so age + 1 causes a TypeError.

```
print("Hello World")
age = int(input("Enter age: "))
print(age + 1)
```

14. ValueError, because “ten” cannot be converted to an integer.

**15.** Use int(input(...)) as shown above, or validate the input before conversion.

## Worksheet 3: Algorithms, Flowcharts and Coding a Solution

**A.** Average: three marks → add and divide by 3 → average. Pass: marks → compare with 40 → Pass/Try again. Recycling: item name → match rules → bin recommendation.

**B. 1.** START → Input A and B → If A > B, display A; otherwise display B → END.

**2.** START → Input number → If number % 2 == 0, display Even; otherwise display Odd → END.

**C. 3.** Accept a flowchart with input, decision diamond, two output paths and end.

**D. 4.** Errors: input returns a string, so convert with float(); missing colons; missing indentation.

```
temperature = float(input("Enter temperature: "))

if temperature < 20:
    print("Cool")
elif temperature <= 30:
    print("Warm")
else:
    print("Hot")
```

**E. 5.** See Program 4 in Unit D. Award marks for correct rules, algorithm, indentation and testing.

## MCQ Answer Key

1. (b)
2. (b)
3. (b)
4. (b)
5. (b)
6. (b)
7. (b)
8. (b)
9. (c)
10. (c)
11. (c)
12. (b)
13. (b)
14. (b)
15. (b)
16. (b)
17. (a)
18. (a)
19. (b)
20. (b)
21. (b)

22. (b)

23. (b)

24. (b)

25. (b)

**Marking:** 1 mark per correct answer; total 25 marks.

## Quick Quiz Answers

1. It gives instructions to a computer. 2. # 3. It displays output. 4. A named container that stores a value. 5. Float. 6. A string. 7. if. 8. It repeats code a known number of times. 9. Finding and fixing errors. 10. No; it is rule-based unless it uses a trained model.

## Assessment Marking Scheme

**Section A:** 1-a, 2-a, 3-a, 4-b, 5-a. (5 marks)

**Section B:** 1. An algorithm is a step-by-step method to solve a problem. 2. Pseudocode is a simple English-like plan for a program. 3. A syntax error breaks Python grammar; a logic error runs but gives a wrong result. 4. A variable stores a value; e.g., age = 13. 5. A rule-based program follows programmer-written rules; machine learning learns patterns from data. (2 marks each)

**Section C:** 1. 12 / True. (3 marks) 2. Add colon and indent:

```
if 5 > 3:
    print("Yes")
```

3.

```
temperature = float(input("Enter temperature: "))
if temperature < 20:
    print("Cool")
elif temperature <= 30:
    print("Warm")
else:
    print("Hot")
```

**Section D:** 1.

```
item = input("Enter item: ").lower()
if item == "plastic bottle":
    print("Use the recycling bin.")
elif item == "paper":
    print("Use the paper bin.")
elif item == "food waste":
    print("Use the compost bin.")
else:
    print("Ask an adult.")
```

2. Logic error. Testing with several inputs helps find rules that produce wrong results. (2 marks)

**Total: 30 marks.**

## Puzzle Solutions

Puzzle 1: See locations above. Puzzle 2: 1-B, 2-C, 3-A, 4-E, 5-D. Puzzle 3: 12, True, Score: 7. Puzzle 4: Convert input to int, add colons, indent print statements.

## Activity Expected Outcomes

Activity 1: Students understand precise sequencing. Activity 2: Students create a simple input-output program. Activity 3: Students identify input, process and output. Activity 4: Students correct syntax, indentation and logic errors. Activity 5: Students design and test a rule-based decision system.